

Assignment 3

DUE DATE: 11:59PM, MONDAY MAY 25TH 2026 (MELBOURNE TIME)

1 Introduction

The assignment is worth 20% of your total mark and is done in pairs (the same pairs as assignments 1 and 2, unless you have been re-allocated to a new pair).

The aim of this assignment is to use the SPARK Ada toolset to implement, specify, and formally verify a 2D particle simulator with collision avoidance.

You are given a simulator in which two circular objects move with constant velocities inside a rectangular arena, bouncing off the walls. The provided `simulation.html` file shows a reference simulation produced by the completed solution (see Figure 1). Without collision detection, the objects would collide with each other; the completed solution detects this and halts the simulation before it happens.

Your job is to:

1. Implement the core simulation engine.
2. Write formal specifications for the engine you implemented.
3. Implement and formally verify a future collision detection system that halts the simulation before any collision occurs.
4. Prove the soundness of that system using SPARK — that is, prove that if the simulation continues past your check, no collision is possible on the current frame.

As usual, get started early and use your pair to maximum advantage.

Make sure you can build and run the provided code (see Section 3.0) ASAP!

2 The Simulator

2.1 Overview

The simulator models two circular objects moving inside a rectangular arena. Each object has a position, a velocity, and a radius. On each *tick*, every object's position is advanced by its velocity. When an object reaches a wall (i.e. its position plus or minus its radius exceeds an arena boundary), its velocity component perpendicular to that wall is negated (a “bounce”).

The simulation uses exact arithmetic (`Big_Real`) rather than floating-point numbers. This is critical: it means the SPARK prover can reason algebraically about positions and velocities without floating-point unsoundness.

The `Big_Real` type in Ada holds *arbitrary precision rational numbers*. For code that never deals with irrational numbers, these behave exactly like mathematical real numbers. Modern Ada



Figure 1: The provided `simulation.html` opened in a browser, showing two objects (red and green) moving inside the arena. Use the playback controls to step through frames and observe collisions.

allows `Big_Real` to be used in both specifications and code. (Older Ada toolchains like GNAT 2019 allowed `Big_Real` to be used only in specs.)

2.2 Provided Code

You are provided with the following files.

2.2.1 Given complete (do not modify)

- `collision_math.ads` / `collision_math.adb` — The mathematical core (see Appendix A for an explanation of the underlying mathematics). Provides:
 - `Will_Collide_Vec`: an expression function that, given a position difference S , a velocity difference V , and a squared separation threshold `Eps2`, returns whether the two trajectories will ever come within that distance.
 - `Sq_Dist_At_Vec`: squared distance between two trajectories at time T .
 - `Check_Implies_Safe_Vec`: a ghost lemma proving that if `Will_Collide_Vec` says “no collision” then `Sq_Dist_At_Vec` $>$ `Eps2` for all $T \geq 0$.
 - `Lemma_Sq_Dist_Bridge`: a ghost lemma that bridges actual positions (expressed as initial position + velocity $\times$ time) to the squared-distance formula.

This package is fully proved. You should treat it as a black box: you need only its contracts, not its internal proofs.

- `vector.ads` / `vector.adb` — A 2D vector type with `Add`, `Sub`, `Scale`, and `Dot`, all as expression functions over `Big_Real`.
- `spatial.ads` — Position and velocity types derived from `Vector.Vector`, with conversion functions (`To_Position`, `To_Velocity`, `To_Vector`, `Vel_To_Vector`), `Move`, `Pos_X`, `Pos_Y`, `Negate_Vel_X`, `Negate_Vel_Y`. All are expression functions visible to the prover.
- `display.ads` / `display.adb` — HTML visualisation output. You do not need to modify or understand this package.
- `simulation.html` — A pre-generated reference simulation produced by the completed solution, including collision detection. The simulation stops early when a future collision is detected. You can use this to verify your Universe implementation (Task 3.2): the initial frames should match even though your version will run longer (past the point where collisions occur). Once you have completed Task 3.5, your output should match the reference exactly.

2.2.2 Given as skeletons (you fill in the holes)

- `universe.ads` — Generic package parameterised by `Max_Items`. The type declarations, public API signatures, and private record types are provided. Procedure bodies and postconditions are left for you to write.
- `universe.adb` — Skeleton with stub procedure bodies for you to implement. The `Print` procedure (`SPARK_Mode => Off`) is provided.
- `main.adb` — The main simulator. The simulation loop structure, arena constants, initial positions/velocities, display capture/save, bounce handling (`Detect_Bounces`, `Reflect_Velocity_X/Y`, `Reset_Universe`, `Print_Collision`), and the helper functions `Squared_Dist` (squared distance between two items) and `Pair-Sep2` (squared collision threshold for two items) are all provided. What is *not* provided: collision detection, the safety proof, the position invariant, and the Universe postconditions.

Do not modify any of the provided files that are marked “do not modify” above. You should only modify `universe.ads`, `universe.adb`, and `main.adb`.

2.3 Architecture Notes

A few design decisions in the provided code may not be immediately obvious:

- **Why `Big_Real`?** Exact arithmetic means the prover can reason algebraically. Floating-point arithmetic would introduce rounding errors that make formal verification impractical.
- **Why does `Reset_Universe` exist?** After a bounce, an object’s velocity changes. The position invariant (which you will define in Task 3.4) ties each object’s position to its initial position plus velocity times the tick count. After a velocity change, this relationship no longer holds. `Reset_Universe` re-initialises the universe with the current positions as the new initial conditions and resets the tick count to zero, thereby re-establishing the invariant.

- **Why is collision detection separate from bouncing?** The bounce logic modifies the universe (via `Reflect_Velocity`). The collision check depends only on `Initial_Positions`, `Initial_Velocities`, and `Initial_Radii` — not on the universe’s intermediate states during bounce handling.

3 Your Tasks

Get started early.

Total marks: 20.

3.0 Task 0: Download and Build

This assignment uses the same Ada tooling (GNAT, Alire, VS Code with a devcontainer) that you set up in Workshop 5. If you have not completed that workshop, do so first.

Download the provided code from the LMS and open it in VS Code as a devcontainer (the same way you did for the workshop code). Build the code from the devcontainer terminal:

```
$ alr build
```

To run the SPARK prover (which you will need from Task 3.3 onwards):

```
$ gnatprove -P main.gpr src/main.adb --mode=prove
```

The code will compile at this stage, but will crash at runtime because the Universe procedure bodies are stubs. You should see output like:

```
raised PROGRAM_ERROR : Init not yet implemented
```

This is expected — you will replace these stubs with real implementations in Task 3.2.

3.1 Task 1: Code Understanding (2 marks)

Read the provided `spatial.ads`, `vector.ads`, and `universe.ads` files, then answer the following two questions. Submit your answers as comments at the top of your `main.adb` file.

1. The `Spatial` package defines separate types `Position` and `Velocity`, both derived from `Vector.Vector`. Explain what advantage this gives over using `Vector.Vector` directly for both positions and velocities throughout the codebase. What kinds of programming errors does Ada’s type system prevent by having distinct types? Give a concrete example of a mistake that would be caught at compile time with separate types but would silently compile if everything were a `Vector`.
2. Several procedures in `universe.ads` have preconditions. For each precondition, explain why it is needed: what runtime error could occur if the precondition were removed?

3.2 Task 2: Implement Universe (3 marks)

Implement the procedure bodies in `universe.adb`:

- **Init** — Initialise the universe with zero items.
- **Add_Item** — Append an item with the given position, velocity, and radius.
- **Tick** — Advance each item's position by its velocity (using `Spatial.Move`).
- **Reflect_Velocity_X** — Negate the X component of an item's velocity (using `Spatial.Negate_Vel_X`).
- **Reflect_Velocity_Y** — Negate the Y component of an item's velocity (using `Spatial.Negate_Vel_Y`).

Verification. Build and run the simulator. It should produce a `simulation.html` file. Open it in a browser and compare it with the provided reference `simulation.html` — you should see two objects bouncing around the arena. Collisions will still occur; that is expected at this stage.

Note. The `Tick` procedure requires a loop. You may find it helpful to start thinking about what loop invariant this loop will need (tracking which items have been processed and which have not), as you will need one in Task 3.3.

3.3 Task 3: Universe Specifications (4 marks)

Write postconditions for `Init`, `Add_Item`, and `Tick` in `universe.ads`. Your postconditions should fully specify the effect of each procedure — that is, they should describe everything that changes *and* assert that everything else is preserved. Study the provided postconditions on `Reflect_Velocity_X` and `Reflect_Velocity_Y` for the expected level of detail.

Verification. Run `gnatprove` on `main.adb` (since `Universe` is a generic package, `gnatprove` cannot analyse it directly — it analyses the instantiation in `Main`). All postconditions should be proved.

Hint: you will need a loop invariant for `Tick` to prove its postcondition.

3.4 Task 4: Position Invariant (4 marks)

In this task you will define an invariant that precisely characterises the state of the universe at every point during the simulation, and prove that the main loop preserves it.

3.4.1 Deriving the invariant

Consider how the simulation works:

- `Initial_Positions` holds the objects' positions either from the start of the simulation or from the most recent bounce (whichever is later). Similarly, `Initial_Velocities` holds their velocities at that same point.
- `Tick_Count` counts the number of ticks since the simulation started or since the last bounce.

Given these observations, derive an equation that describes the expected position of item I at the current moment, in terms of `Initial_Positions(I)`, `Initial_Velocities(I)`, and `Tick_Count`.

Define `Position_Invariant` as an expression function in `main.adb` that encodes this equation. Your invariant should also assert:

- The number of items in the universe is 2.
- `Tick_Count` is non-negative.

Hint: your invariant may also need to say something about each object's velocity and radius — think about what quantities the prover needs to know remain unchanged between bounces.

3.4.2 Add loop invariants and postconditions

Add `Position_Invariant(U)` as a loop invariant of the main simulation loop (you may need additional loop invariants beyond `Position_Invariant` itself).

Write a postcondition for `Reset_Universe` that is strong enough to re-establish `Position_Invariant` after a bounce.

Verification. Run `gnatprove` with `--level=2`. All loop invariants should be proved — both initialisation (the invariant holds on entry to the loop) and preservation (if the invariant holds at the start of an iteration, it holds at the end).

If the prover is unable to discharge some of your proof obligations at the default level, try `--level=2` — the algebraic reasoning involved sometimes needs more time.

3.5 Task 5: Future Collision Detection (3 marks)

In this task you will implement a collision detection system that predicts whether the two objects will ever collide on their current trajectories, and halts the simulation if so. You should read Appendix A before starting this task — in particular, you will need to understand why `Will_Collide_Vec` takes *difference* vectors (position difference and velocity difference) rather than individual positions and velocities.

3.5.1 Define `No_Future_Collision_Pair`

The skeleton provides `Pair_Sep2(I, J)`, which computes the squared minimum separation distance between two objects: $(\text{Initial_Radii}(I) + \text{Initial_Radii}(J))^2$. Two objects are “colliding” when the distance between their centres is at most the sum of their radii, so this squared sum is the threshold for the squared-distance check.

Define an expression function `No_Future_Collision_Pair(I, J)` that returns `True` when `Will_Collide_Vec` (from `Collision_Math`) says the two objects will *not* collide. Apply it to the position difference and velocity difference between items I and J , with threshold `Pair_Sep2(I, J)`.

3.5.2 Add collision checks

- Before the main loop: check whether the initial trajectories will collide. If so, abort the simulation (return from `Main`).
- After each bounce (inside the bounce-handling block): check whether the post-bounce trajectories will collide. If so, exit the loop.
- Add `No_Future_Collision_Pair(1, 2)` as a loop invariant of the main simulation loop.

Verification. Run `gnatprove`. The `No_Future_Collision_Pair` loop invariant should be proved:

- Initialisation: from the pre-loop check (if the check fails, we return before the loop, so on entry to the loop the invariant holds).
- Preservation: on non-bounce iterations, the invariant is trivially preserved because `Initial_Positions` and `Initial_Velocities` do not change. On bounce iterations, it is re-established by the post-bounce check (or we exit the loop).

3.6 Task 6: Soundness Proof (3 marks)

In this final task you will prove that the collision detection system from Task 3.5 is *sound*: that is, if the simulation continues past the collision check, then the two objects are provably separated on the current frame. The proof chain is described at the end of Appendix A. Your job is to encode that proof chain as a ghost lemma procedure that the SPARK prover can use to establish collision freedom on every frame.

3.6.1 Write the soundness lemma

Write a ghost procedure `Lemma_No_Collision_Pair` in `main.adb` that:

- Takes parameters U (the universe), I and J (item indices).
- Has preconditions:
 - `Position_Invariant(U)`
 - I and J are in range ($1 \leq I \leq 2, 1 \leq J \leq 2$)
 - `Tick_Count >= To_Big_Real(0)`
 - `No_Future_Collision_Pair(I, J)`
- Has postcondition: `Squared_Dist(U, I, J) > Pair_Sep2(I, J)`.
- In its body, uses the ghost lemmas from `Collision_Math` to establish the postcondition.

3.6.2 Assert collision freedom

In the main loop, after the loop invariants, add a `pragma Assert` asserting that `Squared_Dist(U, 1, 2) > Pair_Sep2(1, 2)`. This is the property you ultimately want to prove: that the objects are separated on every frame.

If you run `gnatprove` now, the assertion will fail — the prover does not have enough information to discharge it on its own. To help the prover, call your ghost lemma procedure immediately before the assertion. This “teaches” the prover the facts it needs.

Verification. Run `gnatprove`. With the lemma call in place, the assertion should now be proved, establishing that on every frame the simulation runs, the two objects are provably separated — no collision is possible.

3.7 Task 7: Reflection (1 mark)

When the collision detection system predicts a future collision, the simulation halts early.

In a short paragraph (no more than half a page), answer the following: does your proof (Task 3.6) guarantee that when the simulation halts early, a collision would *definitely* have occurred had the simulation continued? Or could the simulation halt even though no actual collision was going to happen? Explain your reasoning, referring to what your proof does and does not establish.

Submit your answer as a comment at the top of your `main.adb` file (alongside your answer to Task 3.1).

4 Submission

You should create a single zip file containing just your modified `universe.ads`, `universe.adb`, and `main.adb` files. Use comments at the top to indicate the names of *all* authors in your pair.

Your code and proofs should build and run against the original provided packages and supporting code.

Late submissions Late submissions will attract a penalty of 2 marks for every day that they are late. If you have a reason that you require an extension, email Toby *well before the due date* to discuss this.

Please note that having assignments due around the same date for other subjects is not sufficient grounds to grant an extension. It is the responsibility of individual students to ensure that, if they have a cluster of assignments due at the same time, they start some of them early to avoid a bottleneck around the due date. The content required for this assignment was presented before the assignment was released, so an early start is possible (and encouraged).

5 Use of Generative AI

You are permitted to use Generative AI tools to assist you with this assignment. However, you should be aware of the following:

- **The final exam will require you to read and write SPARK Ada, including loop invariants and contracts.** The purpose of this assignment is to build your understanding of formal specification and verification in SPARK Ada. If you rely on Generative AI to produce your code without understanding what it does and why, you will be at a significant disadvantage in the exam.
- **You remain responsible for the content of your submission.** Any errors that a Generative AI introduces are *your* errors. SPARK Ada has subtle semantics (e.g. **and then** vs **and** in contracts, **Big_Real** arithmetic, ghost code restrictions) that current LLMs frequently get wrong. You must understand and verify any code you submit.
- **Use gnatprove to validate AI-generated code.** If you use Generative AI to help produce SPARK annotations, do not simply accept its output. Run **gnatprove** to check whether the prover can discharge your proof obligations. If it cannot, something is wrong — either the code, the specifications, or both — regardless of how plausible they look. Note that **gnatprove** succeeding is necessary but not always sufficient: a postcondition that is trivially true (e.g. because it is too weak) will prove but will not earn full marks.
- **Written answers require genuine understanding.** Task 3.1 and Task 3.7 ask you to explain design decisions and reasoning in your own words. Generic or superficial answers that do not engage with the specifics of the provided code will not score well.
- **Document your use of Generative AI.** If you use Generative AI, add a brief comment to the top of your `main.adb` file describing how you used it, and critically, how you validated that the AI-generated code was correct (e.g. what **gnatprove** told you and what you changed as a result).

6 Academic Misconduct

The University misconduct policy applies to this assignment. Students are encouraged to discuss the assignment topic, but all submitted work must represent the pair's understanding of the topic.

The subject staff take plagiarism very seriously. In the past, we have successfully prosecuted several students that have breached the university policy. Often this results in receiving 0 marks for the assessment, and in some cases, has resulted in failure of the subject.

A The Mathematics of Collision Detection

This appendix explains the mathematics behind `Collision_Math`. You do not need to understand the proofs inside `collision_math.adb`, but you *do* need to understand the ideas in this section to use `Will_Collide_Vec` correctly in Task 3.5 and Task 3.6.

A.1 Reducing two trajectories to one

Between one bounce and the next, each object i has an initial position s_i and a constant velocity v_i . At time t (measured from the most recent bounce, or from the start of the simulation) its position is

$$l_i(t) = s_i + t v_i.$$

We want to know whether the two objects will ever come within distance ε of each other, i.e. whether there exists $t \geq 0$ such that $\|l_1(t) - l_2(t)\| \leq \varepsilon$. (For circular objects of radii r_1 and r_2 , a collision occurs when the distance between their centres is at most $r_1 + r_2$, so we set $\varepsilon = r_1 + r_2$.)

The key insight is that the *difference* between the two trajectories is itself a single linear trajectory:

$$l_1(t) - l_2(t) = \underbrace{(s_1 - s_2)}_S + t \underbrace{(v_1 - v_2)}_V.$$

Define:

$$S := s_1 - s_2 \quad (\text{position difference}), \quad V := v_1 - v_2 \quad (\text{velocity difference}).$$

Then the question becomes: does the single trajectory $L(t) = S + tV$ ever come within distance ε of the origin?

This is exactly what `Will_Collide_Vec(S, V, Eps2)` checks, where `Eps2` = ε^2 .

What this means for your code. When you call `Will_Collide_Vec`, you must pass:

- **S**: the *difference* between the two objects' positions (not a single object's position);
- **V**: the *difference* between their velocities;
- **Eps2**: the *squared* sum of their radii.

A.2 How the check works (for intuition)

Working with squared distances avoids square roots. Define

$$f(t) = \|S + tV\|^2 = (S \cdot S) + 2t(S \cdot V) + t^2(V \cdot V).$$

This is a quadratic in t (or a constant if $V = 0$). We want to know whether $\min_{t \geq 0} f(t) \leq \varepsilon^2$.

There are three cases:

1. $V \cdot V = 0$ (so $V = 0$ and objects have identical velocities, so their separation is constant). Check $S \cdot S \leq \varepsilon^2$.
2. $V \cdot V \neq 0$ **but** $S \cdot V \geq 0$ (the objects are not approaching each other). The minimum of f for $t \geq 0$ occurs at $t = 0$, so check $S \cdot S \leq \varepsilon^2$.

3. $V \cdot V \neq 0$ **and** $S \cdot V < 0$ (the objects are approaching). The unconstrained minimum of the parabola¹ occurs at $t^* = -\frac{S \cdot V}{V \cdot V} > 0$. Substituting gives the minimum squared distance:

$$f(t^*) = S \cdot S - \frac{(S \cdot V)^2}{V \cdot V}.$$

Check whether $f(t^*) \leq \varepsilon^2$.

The implementation in `Will_Collide_Vec` combines cases 1 and 2 (since they have the same check) and avoids division in case 3 by multiplying both sides of the inequality by $V \cdot V$:

$$S \cdot S \cdot (V \cdot V) - (S \cdot V)^2 \leq \varepsilon^2 \cdot (V \cdot V).$$

A.3 The two lemmas

The `Collision_Math` package provides two ghost lemmas that you will use in Task 3.6:

- `Check_Implies_Safe_Vec(S, V, Eps2, T)`: if `Will_Collide_Vec(S, V, Eps2)` is `False`, then $f(T) = \|S + TV\|^2 > \varepsilon^2$ for any $T \geq 0$. That is, “no predicted collision” implies “safely separated at all future times.”
- `Lemma_Sq_Dist_Bridge`: connects the abstract formula $f(T)$ to actual positions in the universe. If item i ’s position is $Init_i + Vel_i \times T$, then the squared distance between the two items equals $f(T)$ applied to $S = Init_1 - Init_2$ and $V = Vel_1 - Vel_2$.

Together, these two lemmas form a proof chain: the collision check (`Will_Collide_Vec`) guarantees safety in the abstract formula; the bridge lemma connects the abstract formula to the actual positions; therefore the actual objects are separated.

¹The minimum of a parabola $f(x) = ax^2 + bx + c$ for $a > 0$ is at $x = -\frac{b}{2a}$. In this case $a = V \cdot V$ and $b = 2S \cdot V$, hence the minimum occurs at $t^* = -\frac{S \cdot V}{V \cdot V}$.